



Tarea 1: Micro-Eventos

https://github.com/Grupo8-Pruebas-de-software/Tarea_1

Fecha: 10/09/2025

Profesor: Oscar Reyes

Integrantes

-Felipe Campaña

-Luciano Yevenes

Grupo: 8



Especificación Requerimientos:

1. Autenticación

- El sistema debe permitir iniciar sesión con **usuario (correo)** y **contraseña**.
- El sistema debe permitir crear un usuario con su nombre, correo y contraseña.
- Solo usuarios autenticados podrán ingresar a **crear, editar, eliminar eventos y registrar ventas/devoluciones**.
- Las contraseñas deben almacenarse en la base de datos de manera encriptada.

2. CRUD de Eventos

- Se debe poder **crear un evento**, ingresando los siguientes datos obligatorios:
 - Nombre (string, máximo 100 caracteres).
 - Descripción (texto).
 - Fecha (formato YYYY-MM-DD, debe ser $\geq$ fecha actual).
 - Categoría (opciones predefinidas: Charla, Taller, Show).
 - Precio (entero ≥ 0).
 - Cupos disponibles (entero ≥ 0).
- Se debe poder **consultar** la lista de eventos y el detalle de un evento específico.
- Se debe poder **editar** cualquier campo de un evento existente.
- El evento solo puede ser modificado por el usuario que lo creo.
- Se debe poder **eliminar** un evento.

3. Gestión de Entradas

- Se debe poder **registrar una venta**, lo que resta 1 cupo al evento seleccionado.
- Si el evento ya no tiene cupos (cupos = 0), no se debe permitir registrar nuevas ventas.



- Se debe poder **registrar una devolución**, lo que suma 1 cupo al evento.
- Una devolución no puede aumentar los cupos por encima del valor inicial definido en la creación del evento.

4. Filtrado y Búsqueda

- El sistema debe permitir **filtrar** eventos por:
 - Categoría.
 - Fecha (ejemplo: próximos 7 días).
 - Estado (disponible / agotado).
- El sistema debe permitir **buscar** por palabras clave en nombre o descripción.

5. Reportes

- El sistema debe mostrar un **resumen general** que incluya:
 - Total de eventos registrados.
 - Total de cupos disponibles en todos los eventos.
 - Cantidad de eventos agotados (cupos = 0).
- El reporte debe actualizarse automáticamente al crear, editar o eliminar eventos, y al registrar ventas/devoluciones.

6. Supuestos Claros

- Solo habrá un rol de usuario administrador (no se diferenciarán perfiles).
- No se gestionarán datos de clientes (solo número de entradas vendidas/devoluciones).
- No se requiere integración con pasarelas de pago, las ventas se registran manualmente.
- No se requiere control de múltiples usuarios concurrentes (se asume bajo tráfico).



¿Cómo especificar mejor el requerimiento?

- Se revisaron los requisitos entregados y se **complementaron con supuestos** para que no quedaran dudas.
- Se definieron con mayor detalle los **campos obligatorios** de un evento (nombre, descripción, fecha $\geq$ actual, categoría fija, precio entero ≥ 0 , cupos ≥ 0).
- Se establecieron **reglas claras para las ventas y devoluciones** (no permitir ventas si cupos = 0, ni devoluciones que superen los cupos iniciales).
- Se aclaró que la aplicación requiere **inicio de sesión** y que solo los usuarios autenticados pueden operar con eventos y entradas.

¿Cómo asegurar que el programa cumpla el requerimiento?

- **Pruebas unitarias:** funciones críticas como crearEvento(), venderEntrada(), devolverEntrada() y generarReporte() serán probadas con datos correctos e incorrectos.
- **Pruebas funcionales:**
 - Intentar crear un evento con fecha pasada (debe rechazarlo).
 - Vender entradas hasta agotar cupos (no debe permitir más ventas).
 - Devolver más entradas que las iniciales (debe rechazar).
 - Ingresar al sistema sin iniciar sesión (debe bloquear CRUD y ventas).
 - No permitir que se sobrevendan las entradas (cupos < 0)
- **Pruebas de integración:** comprobar que el resumen de reportes se actualice automáticamente después de ventas, devoluciones o cambios en eventos.
- **Pruebas de interfaz:** verificar que la búsqueda y filtrado devuelvan resultados correctos.
- **Pruebas de seguridad:** verificar que contraseñas no se almacenen en texto plano.



Organización, explicar cómo se organizó el proyecto y el flujo de trabajo de éste.

Para la organización de la tarea se creó un repositorio de Github dentro de una organización para guardar los avances de la tarea. Se aplicó el flujo de trabajo Gitflow, teniendo una rama main que tendría los cambios finales, y otra rama llamada develop donde se fueron incorporando las funcionalidades que íbamos creando.

En la primera etapa se creó en conjunto el esqueleto de la aplicación y definimos la BD que utilizaríamos, junto a las tablas y columnas. Luego el desarrollo se dividió por funcionalidades, para poder trabajar en paralelo:

- Felipe realizó el CRUD de eventos, la venta/devolución de entradas, y el filtrado de eventos.
- Luciano se encargó del registro/inicio de sesión, reporte de eventos y logs del programa.

Cada nueva funcionalidad se creaba a partir de la rama de develop, y una vez completa se enviaba una pull request a develop. Permitiendo que cada integrante pueda revisar y validar el trabajo del otro. Una vez completo el desarrollo se pasó a main los cambios.

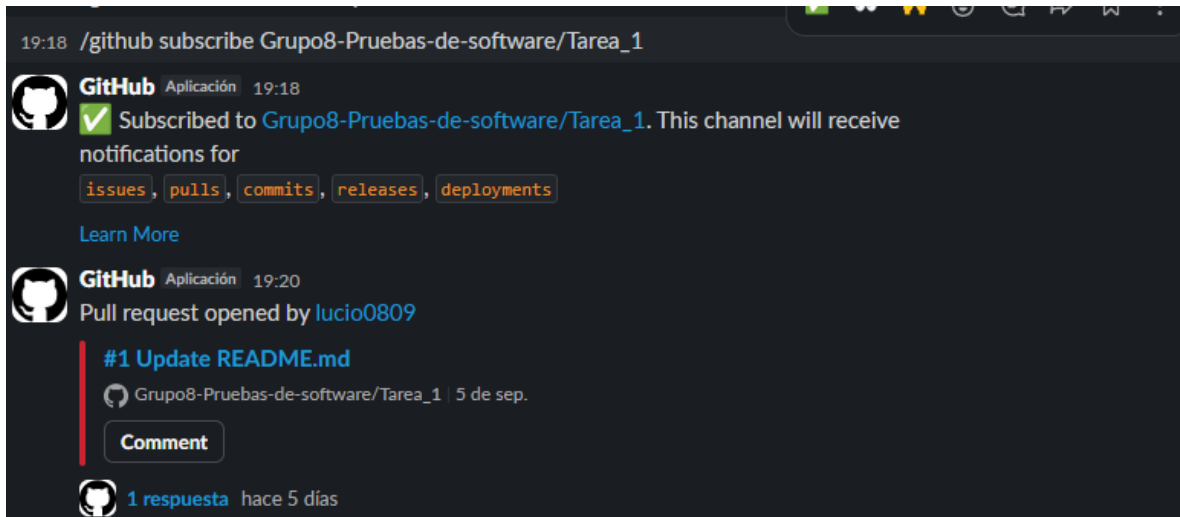
Respecto a las pruebas, esta se conformó de 2 ciclos:

1. En el primer ciclo, cada integrante realizó pruebas individuales, donde se probaron las funcionalidades que el otro integrante realizó.
2. En el segundo ciclo se validaron casos en los que combinaran las diferentes funcionalidades del sistema.

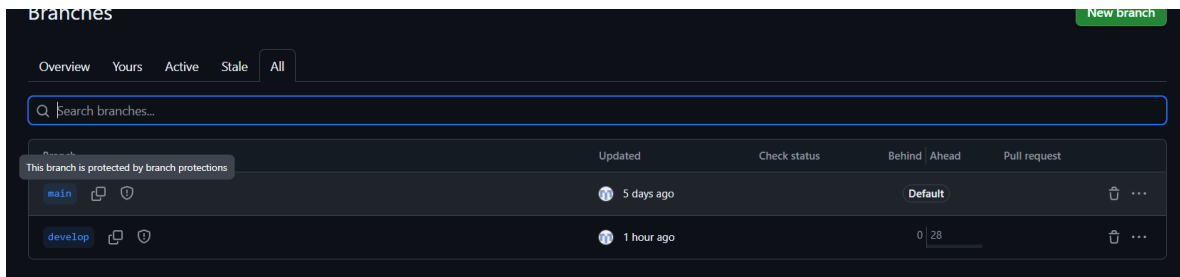


Evidencia de flujo de trabajo y configuraciones realizadas

Integración de Github con Slack:



Protección de ramas:





Pull-Request realizados:

0 Open	8 Closed
Fix inicio sesion	
#9 by Petou21 was merged 1 hour ago • Review required	
Add opcion reports	
#8 by lucio0809 was merged yesterday • Review required	
Filtrado de busqueda	
#7 by Petou21 was merged 3 days ago • Review required	
Add logs	
#6 by lucio0809 was merged 3 days ago • Approved	
Gestor de entradas	
#5 by Petou21 was merged 4 days ago • Review required	
feat: implementar CRUD de eventos en el backend y frontend	
#4 by Petou21 was merged 4 days ago • Review required	
Crear login	
#3 by lucio0809 was merged 4 days ago • Review required	
Update README.md	
#1 by lucio0809 was closed 5 days ago • Review required	

Revisor/aprobador -> Felipe Campaña:

Add logs #6

Merged Petou21 merged 2 commits into `develop` from `add_logs` 3 days ago

Conversation 0 Commits 2 Checks 0 Files changed 5

lucio0809 commented 3 days ago

No description provided.

lucio0809 added 2 commits 3 days ago

- chore(add config for logs)
- chore(create logs for operations)

lucio0809 requested a review from Petou21 3 days ago

lucio0809 changed the base branch from `main` to `develop` 3 days ago

Petou21 approved these changes 3 days ago

Petou21 merged commit `ce1c1f2` into `develop` 3 days ago

Petou21 deleted the `add_logs` branch 3 days ago

Problemas encontrados y como se solucionaron.

Tuvimos un problema al momento de implementar slack con github, el cual no mostraba los commits de otras ramas. Esto se soluciono con un comando que encontramos.



En general no tuvimos problemas que nos complicaran realizar la tarea.

Pruebas.

¿Cómo vamos a probar?

- Se elaboro un listado de casos de prueba que cubrían las funcionalidades realizadas. Los casos se ejecutaron y se hizo el registro en la plataforma greentest.ai

¿Quién prueba qué?

- Cada integrante probo las funcionalidades que realizo el otro integrante.

¿Haremos pruebas cruzadas?

- Si, en el primer ciclo se realizaron pruebas individuales, pero en el según ciclo se hicieron las pruebas cruzadas verificando el funcionamiento entre sí de las diferentes funcionalidades realizadas.